shared memory & semaphors

תקשורת בין תהליכים - IPC - Inter Process Communication

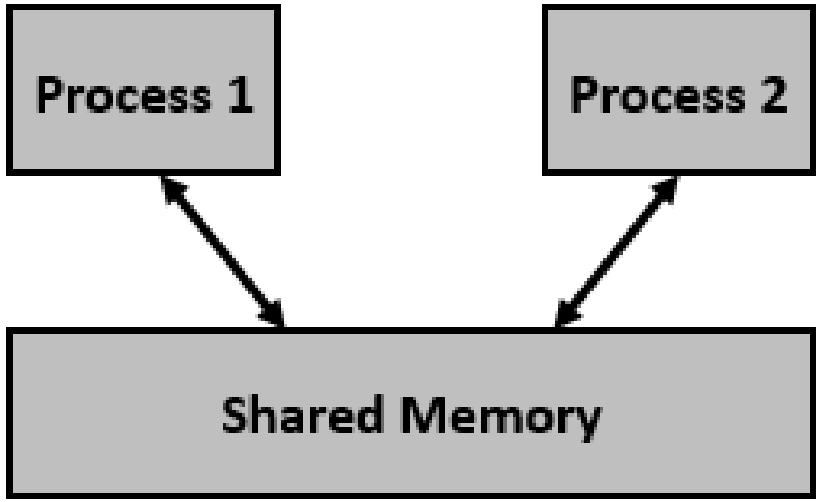
- דרכי תקשורת בין תהליכים –

1. זיכרון משותף
2. תור הודעות
3. סמפורים

- כל השלושה זקוקים ל"מפתח" ייחודי במערכת בשביל לקיים תקשורת תקינה.

זיכרון משותף

- מאפשר לתהליכים שחולקים אותו לפנות לאותו קטע זיכרון.
- לדוגמא – מערך. מתן אפשרות לקרוא/לכתוב לכל תא במערך.



זיכרון משותף – יצירת מפתח משותף FTOK

- ליצירת מפתח ייחודי לשימוש בזיכרון המשותף נשתמש בפונקציה

```
key_t ftok(const char *pathname, int proj_id);
```

- הפונקציה מקבלת שם קובץ/נתיב לקובץ מסוים ו integer ויודעת לייצר מפתח ייחודי שבו ישתמשו כל התהליכים שירצו גישה לזיכרון המשותף.

- שימוש בפונקציה זו מחייב להוסיף לתכנית

```
#include <sys/types.h>
#include <sys/ipc.h>
```

- הפונקציה מחזירה : בהצלחה – את המפתח הייחודי שנוצר

בכישלון – ערך מוחזר הוא 1-

זיכרון משותף – יצירת מפתח משותף FTOK

- דוגמא –

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/ipc.h>

void main(){
key_t key = ftok("lab1.c" , 21);
printf("key is %d\n",key);

}
```

- פלט –

```
sysadmin@localhost:~$ ./a.out
key is 361238803
```

- אם נשנה את השם קובץ/המספר – נקבל מפתח חדש.

זיכרון משותף – shmget

- shmget – יצירת סגמנט זיכרון/קבלת מזהה גישה לזיכרון.

```
#include <sys/ipc.h>
#include <sys/shm.h>

int shmget(key_t key, size_t size, int shmflg);
```

- הפונקציה מקבלת כפרמטר:

1. מפתח ייחודי שהחזיר ftok
2. גודל רצוי של זיכרון משותף
3. דגל/דגלים

דגלים שימושיים :

-IPC_CREAT - מייצר מנגנון שיקים את הזיכרון המשותף (אם הוא לא קיים כבר)

-IPC_EXCL – אם המנגנון קיים כבר (מישהו כבר משתמש במנגנון עם מפתח כזה) אז פעולת shmget תכשל.

-אפשר להוסיף דגל שמסמל את ההרשאות למנגנון (כמו ב chmod לדוגמא 0777)

הפונקציה תחזיר: בהצלחה – מזהה מקטע זיכרון משותף shared memory id = shmid

בכישלון - תחזיר 1-.

זיכרון משותף – shmget

- דוגמא

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/shm.h>
void main(){
key_t key = ftok("lab1.c" , 22);
int shmid = shmget(key,128,IPC_CREAT);
printf("key is %d\n , the shmid is %d \n",key,shmid);

}
```

- פלט

```
sysadmin@localhost:~$ ./a.out
key is 361238803
 , the shmid is 32769
```

זיכרון משותף – shmget

סוגי שגיאות נפוצות שעלולות לקרות בעת קריאה לפונקציה –

- ביקשנו גודל זיכרון שגדול/קטן מהמותר.
- המערכת לא מצליחה להקצות זיכרון.
- אין לנו הרשאה להקצאת זיכרון.

זיכרון משותף – shmat

- הפונקציה מצרפת לזיכרון רגיל של כל תהליך את הזיכרון המשותף עם ה shmid שחזר מ shmget.

```
#include <sys/shm.h>

void *shmat(int shmid, const void *shmaddr, int shmflg);
```

- הפונקציה מקבלת:
- shmid שחזר מ shmget.
- הכתובת במקטע זיכרון שאנו רוצים מצביע אליו (בדרך כלל 0 - ניתן למערכת הפעלה להחליט לבד את המיקום)
- דגלים – בדרך כלל 0 – אין מגבלות.

```
char *segmem = shmat(shmid,0,0);
```

- הפונקציה מחזירה – בהצלחה : מצביע אל המקטע של הזיכרון המשותף.

  בכישלון : תחזיר 1-.

זיכרון משותף – shmctl

- סיום עבודה עם זיכרון משותף – שחרור משאבים.

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

- הפונקציה מקבלת:

1. shmid שחזר מ shmget – מזהה הזיכרון אותו נרצה למחוק.
2. cmd – דגל IPC_RMID שמסמן למחוק את המקטע זיכרון משותף.
3. buf - מצביע ל"מבנה" בקובץ <sys/shm.h>. בדרך כלל 0.

```
shmctl(shmid,IPC_RMID,0);
```

הפונקציה מחזירה – בהצלחה : מוחקת את מקטע זיכרון משותף.

בכישלון: מחזירה 1-.

דוגמא – תהליך בן ואבא מתקשרים בעזרת זיכרון משותף

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SEGSIZE 100
main()
{
        key_t key;
        int   shmid;
        char  *segptr;
        int r_pid;
        int status;
        key = ftok("lab1.c", 21); // create key

        if((shmid = shmget(key, SEGSIZE, IPC_CREAT|IPC_EXCL|0666)) == -1){ //create shared memory
                printf("Shared memory segment exists\n");
        }

        if((segptr = shmat(shmid, 0, 0)) == NULL){ // create pointer to shared memory
                perror("shmat");
                exit(1);
        }
        r_pid=fork();
        if(r_pid==0) // child
        {
           printf("child with PID number : %d (parent PID is %d) writes a text to the shared memory\n",getpid(),getppid());
           strcpy(segptr,"This text is written by the child process");
        }
        else // father
        {
            wait(&status); // wait to end of child proccess
             printf("i am  father\n");
              printf("The following text is received by the child process with PID number %d: %s\n",r_pid,segptr);
        }
        shmctl(shmid, IPC_RMID, 0);
}
```

זיכרון משותף - דוגמא

- פלט

```
child with PID number : 860 (parent PID is 859) writes a text to the shared memory
i am  father
The following text is received by the child process with PID number 860: This text is written by the child process
```

זיכרון משותף – 2 תהליכים שונים מתקשרים בעזרת זיכרון משותף

```c
//READ FROM SHARED MEMORY
#include <sys/ipc.h>
#include <sys/shm.h>
# include <stdio.h>
int main() {
    // ftok to generate unique key
    key_t key = ftok("lab1.c",65);

    // shmget returns an identifier in shmid
    int shmid = shmget(key,1024,0666|IPC_CREAT);

    // shmat to attach to shared memory
    char *str = (char*) shmat(shmid,(void*)0,0);
    printf("Data read from memory: %s\n",str);

    // destroy the shared memory
    shmctl(shmid,IPC_RMID,0);
    return 0;
}
```

```c
//WRITE TO SHARED MEMORY
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include<string.h>
int main() {
    // ftok to generate unique key
    key_t key = ftok("lab1.c",65);

    // shmget returns an identifier in shmid
    int shmid = shmget(key,1024,0666|IPC_CREAT);

    // shmat to attach to shared memory
    char *str = (char*) shmat(shmid,(void*)0,0);
    strcpy(str, "hello i write to shared memory");
    printf("Data written in memory is : %s \n" , str);
    return 0;
}
```

זיכרון משותף – 2 תהליכים שונים מתקשרים בעזרת זיכרון משותף

- פלט –

```
sysadmin@localhost:~$ gcc lab1-write.c
sysadmin@localhost:~$ ./a.out
Data written in memory is : hello i write to shared memory
sysadmin@localhost:~$ nano lab1-read.c
sysadmin@localhost:~$ gcc lab1-read.c
sysadmin@localhost:~$ ./a.out
Data read from memory: hello i write to shared memory
```

תרגיל

- כתוב תכנית ובה האבא יוצר בן ושניהם חולקים זיכרון משותף.
- הבן ממלא מערך בגודל 10 במספרים רנדומליים ומדפיס את המערך.
- האבא ממתין שהבן יסיים , קורא את המערך שהבן מילא ומדפיס אותו.

- פלט לדוגמא -

```
 child with PID number : 6080  fill Data to the shared memory
 a[i] : 1804289383
 a[i] : 846930886
 a[i] : 1681692777
 a[i] : 1714636915
 a[i] : 1957747793
 a[i] : 424238335
 a[i] : 719885386
 a[i] : 1649760492
 a[i] : 596516649
 a[i] : 1189641421
i am  father
The following array was filled by my child proccess with pid 6080
 shm_array[i] : 1804289383
 shm_array[i] : 846930886
 shm_array[i] : 1681692777
 shm_array[i] : 1714636915
 shm_array[i] : 1957747793
 shm_array[i] : 424238335
 shm_array[i] : 719885386
 shm_array[i] : 1649760492
 shm_array[i] : 596516649
 shm_array[i] : 1189641421
```

תרגיל

- כתוב 2 תכניות שחולקות ביניהן זכרון משותף.
- תכנית ראשונה כותבת לזיכרון המשותף מערך בגודל 10 עם מספרים רנדומליים.
- תכנית שניה קוראת מהזיכרון מהשותף.

תקשורת בין תהליכים - סמפורים

- מבנה המאפשר תיאום בין תהליכים.
- משותף לכמה תהליכים (בדומה לזיכרון משותף).
- מנגנון לסנכרון מספר תהליכים הפועלים במקביל.
- גורם לכך שפעולות של תהליכים יתבצעו בסדר מסוים.

- סמפור מורכב מ 2 שדות – ערך מספרי ותור של תהליכים הממתינים עליו.
- קיימות 2 פעולות על הסמפור :

1. DOWN – מקטין את המונה ב-1 וממתין עד שערך הסמפור אינו שלילי (שמות נוספים p , wait)
2. UP – מגדיל את ערך המונה ב-1 ומשחרר את אחד התהליכים הממתינים בתור. (שמות נוספים v , signal)

תקשורת בין תהליכים - סמפורים

- קיימים 3 סוגי סמפורים :
- סמפור מונה – ערכו ההתחלתי N והוא שולט על המשאב עם N תהליכים.

- סמפור בינארי - יכול לקבל ערך 0 או 1
ומאפשר לתהליך 1 גישה למשאב.

```
Task A                      Task B
...                         Take BinSemaphore   <== wait for something
Do Something Noteworthy
Give BinSemaphore           do something    <== unblocks
```

```
Thread A                     Thread B
 Take Mutex
   access data
   ...                        Take Mutex  <== Will block
   ...
 Give Mutex                     access data  <== Unblocks
                                ...
                              Give Mutex
```

- mutex - יכול לקבל ערך 0 או 1
ומאפשר גישה בלעדית למשאב.
רק מי שנעל את הmutex יכול לשחרר אותו.
(בסמפור בינארי גם תהליך אחר
יכול לשחרר את הסמפור)

יצירת סמפורים

- נוסיף לתכנית #include<sys/sem.h>

```
int semget(key_t key, int nsems, int semflg);
```

- פונקציה ליצירת קבוצת(מערך) סמפורים

- הפונקציה מקבלת:

1. key – מפתח ייחודי שקיבלנו ע"י הפונקציה ftok.
2. nsems – מספר סמפורים שנרצה ליצור.
3. semflg – דגלים:

-IPC_CREAT - אם לא קיימת קבוצה של סמפורים עם מפתח מסויים – צור אותה.

-IPC_EXCL – אם קבוצת הסמפורים קיימת , הפעולה תיכשל.

- הפונקציה מחזירה: בהצלחה : מספר מזהה של קבוצת סמפורים - semid

בכישלון: תחזיר 1-.

פעולות על סמפורים

```
int semop(int semid, struct sembuf *sops, size_t nsops);
```

- הפונקציה מקבלת:

1. semid – מזהה של קבוצת סמפורים (כתובת בזיכרון).
2. sops – מצביע למערך "מבנים" של הסמפור.
3. nsops – גודל המערך sops.

פעולות על סמפורים – מבנה sops

- יצרנו מערך סמפורים – כל תא מערך (כל סמפור) הוא מסוג "מבנה".

מבנה Sembuf:

```
struct sembuf {
    short sem_num;  // מספר הסמפור
    short sem_op;   // הפעולה לביצוע
    short sem_flg;  // הגדרות נוספות
}
```

sem_num - אינדקס של הסמפור בתוך מערך הסמפורים.

sem_op - פעולה לביצוע על הסמפור.

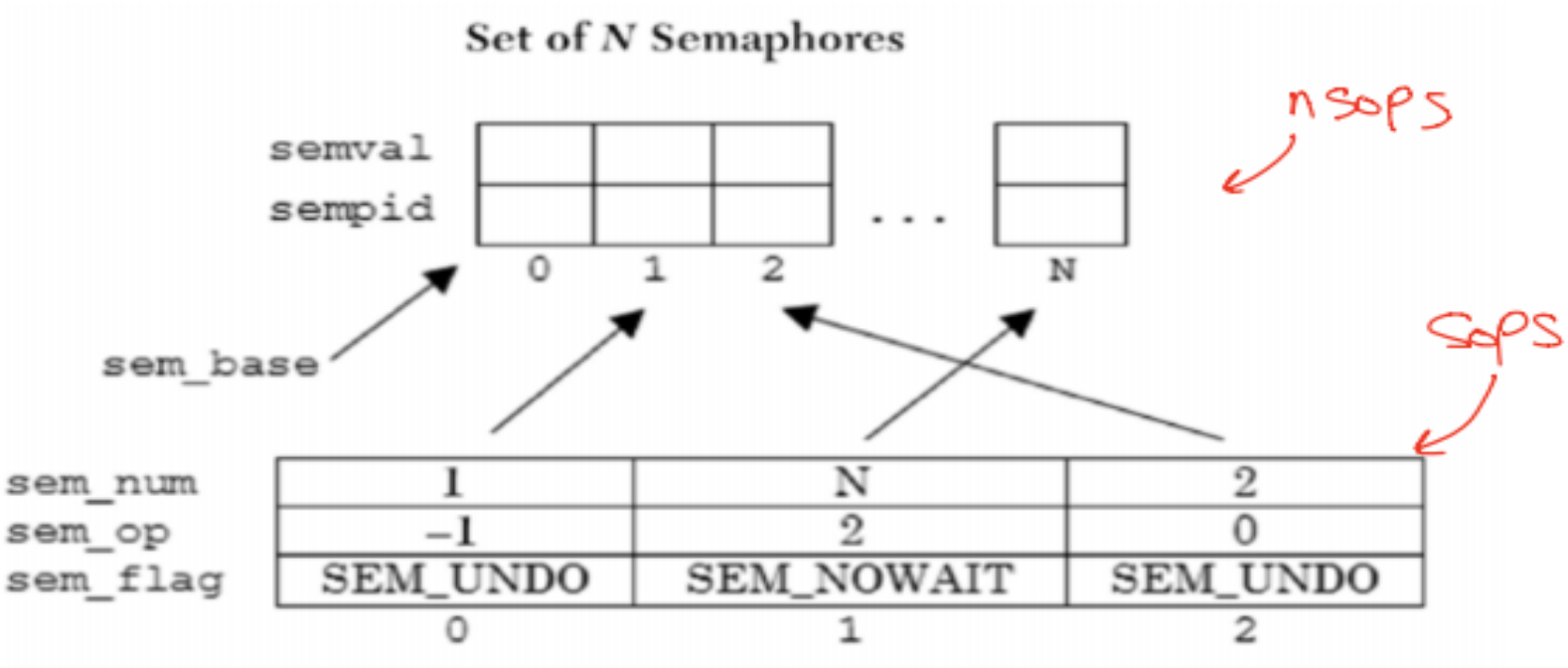
פעולת חשמל $\begin{cases} sem_op = -1 \\ sem_flg = 0 \end{cases}$

פעולת up. $\begin{cases} sem_op = 1 \\ sem_flg = 0 \end{cases}$

sem_flg - פעולה יכולה להכיל דגלים. לדוגמא דגל:

SEM_UNDO - פעולה מתבצעת במקרה בדיקה, מוקמים אם תסתיים בהצלחה.

פעולות על סמפורים – מבנה sops



- הפונקציה מאפשרת לנו לבצע סדרה של פעולות בגודל רצוי על כמה סמפורים.
- הפונקציה מחזירה 0 במידה והצליחה , 1- במידה ונכשלה.

פעולות על סמפורים – דוגמא

*דוגמא לפעולת אתחול קבוצת סמפורים

```
#include <sys/sem.h>
...          //  וכו' semget -ל, ftok -ל קריאה
int semid;
struct sembuf sops[2]; //  מערך מבנים בגודל 2
int nsops=2; //  מערך בגודל 2
int result;

sops[0].sem_num = 0  //  0 סמפור באינדקס
sops[0].sem_op = 1   //  יכול להות 0, 1
sops[0].sem_flg = 0  //  ללא דגל

sops[1].sem_num = 1  //  1 סמפור באינדקס
sops[1].sem_op = 2   //  יכול לגרום 2 תהליכים להפנות
sops[1].sem_flg = 0  //  ללא דגל

result = semop(semid, sops, nsops)
```

מחיקת סמפורים

- חובה למחוק את קבוצת הסמפורים בכל סוף תכנית למען שחרור משאבים.

- הפונקציה מקבלת:

```
int semctl(int semid, int semnum, int cmd, ...);
```

1. semnum /semid – מזהה של קבוצת סמפורים (כתובת בזיכרון) או סמפור ספציפי שנרצה למחוק.
2. cmd – נעביר דגל IPC_RMID – פקודה למחיקה.

מחיקה של קבוצה שלמה

מחיקה של סמפור ספציפי (בדוגמא – אינדקס 0)

```
shmctl(shmid,IPC_RMID,(struct shmid_ds *)0);
semctl( semid, 0, IPC_RMID, 0);
```

- נוכל להעביר גם דגלים נוספים דרך פונקציה זו ולקבל מידע כגון:

GETVAL – ערך של הסמפור.

SETVAL – עדכון ערך סמפור.

```
union _semun arg;
arg.val = 1;
```

```
semctl(semid, 0, SETVAL, arg);
```

הפונקציה מחזירה – בעת הצלחה – מספר חיובי כלשהו , תלוי בדגל שהעברנו. בכישלון 1- .

מטלה

- להריץ דוגמא לשימוש בסמפורים.
- קובץ במודל - semaphore_sample2